

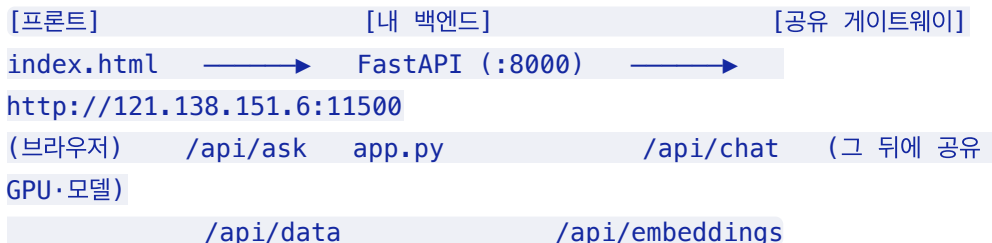
01. 시설팀 — 공간조화

4주차 한 줄 컨셉: “공유 게이트웨이(:11500) 위에서 내 앱을 운영급으로.” 이제 각자 모델을 띄우지 않아도 됩니다. 게이트웨이가 추론 누출(think) 차단·답변 길이 상한·모델명 자동 보정을 한곳에서 처리합니다. 내 앱은 **모델 주소 한 줄만** :11434 → :11500 으로 바꾸면 끝입니다.

※ 이 가이드는 **일반형**입니다. (이 과제는 코드 제출이 없어, 줄번호 대신 “C 조회 과제의 표준 구조”를 기준으로 안내합니다. 본인 코드가 있으면 같은 자리만 찾아 바꾸면 됩니다.)

1. 내 앱 한눈에 — “C 조회 과제는 이런 모양”

- **아키타입: C(조회·검색)** 형. 원내 공간(기계실·세미나실·약제부 등)을 자연어로 묻고, 위치·면적·부서·구분을 찾아주는 조회 챗봇입니다. 3주차에서는 여기에 **임베딩 검색(RAG)** 을 엮어, 키워드가 안 맞아도 의미로 찾아주게 만들었습니다. (예: “물 끓이는 설비 있는 곳?” → 기계실)
- **핵심 기능:** ① 공간 데이터(더미 8칸)에서 질문과 의미가 가까운 조각만 골라 근거로 답하기, ② 답변 아래 출처(근거) 칩 표시, ③ 바이브↔하네스 토크(검색 있음/없음 비교), ④ “면적 합이 가장 큰 부서?” 같은 집계는 백엔드가 직접 계산한 값을 인용(모델이 숫자를 지어 내지 않음).
- **표준 구조 (3단):** 화면과 모델 사이에 **항상 내 백엔드가 끼어 있습니다**. 이게 3주차에서 강조한 “프론트/서버 분리”입니다.



- **왜 이 구조인가:** 브라우저(프론트)는 **절대 모델 서버를 직접 부르지 않습니다**. 모델 호출은 오직 백엔드(`app.py`)만 합니다. 4주차에서 우리가 바꾸는 곳도 **백엔드 한 군데**뿐입니다(프론트는 손대지 않음).

🚫 **여전히 금지:** HTML/자바스크립트에서 모델 주소(:11434 / :11500)나 `ollama` 를 직접 호출하지 마세요(CORS·보안 사고). 프론트는 오직 내 백엔드 (`http://127.0.0.1:8000`)만 부릅니다.

2. 바뀌는 것 — URL 한 줄 (:11434 → :11500)

C 조회 과제의 백엔드(`app.py`)에는 모델 주소가 보통 맨 위 **설정 구역**에 한 줄로 모여 있습니다. 표준형은 이렇게 생겼습니다.

```
# app.py 상단 설정 구역 (표준 예)
OLLAMA_URL = "http://121.138.151.6:11500"      # ← 게이트웨이 (이렇게 두
면 됩니다)
CHAT_MODEL = "qwen3.6:35b-a3b"                # 채팅(답변) 모델 - 그대
로 뒤도 됩니다
EMBED_MODEL = "bge-m3:latest"                  # 검색(임베딩) 모델 - 그
대로
```

원래(3주차) 코드는 끝이 **:11434** 였습니다. 11434는 “각자 자기 PC에서 모델을 직접 띄울 때” 쓰는 raw 포트라서, 생각 과정이 그대로 새어 나오거나 답이 비는 문제가 있었습니다. **포트 숫자만 11500** 으로 고치면 됩니다(IP **121.138.151.6** 은 그대로).

```
- OLLAMA_URL = "http://121.138.151.6:11434"
+ OLLAMA_URL = "http://121.138.151.6:11500"
```

✅ 찾는 요령 (코드를 잘 몰라도)

`app.py` 를 열고 **11434** 를 검색(찾기)하세요. 나오는 자리마다 **11500** 으로 바꾸면 됩니다.

- 채팅 호출: `.../api/chat` 부르는 곳
- 임베딩(검색) 호출: `.../api/embeddings` 부르는 곳
- 상태 점검: `.../api/tags` 부르는 곳 (있을 수도, 없을 수도)

⚠ 주소가 여러 군데 박혀 있을 수 있습니다. C 조회는 채팅(`/api/chat`)과 임베딩(`/api/embeddings`)을 둘 다 쓰므로, **11434** 가 2~3곳에 있을 수 있습니다. **하나도 빠짐없이 전부 11500** 으로 바꾸세요. 한 곳만 고치면 답변은 게이트웨이로 가도 검색은 옛 주소로 가서 절반만 동작합니다.

모델명·옵션은 신경 쓰지 않아도 됩니다. `CHAT_MODEL` 을 어떻게 적어 뒀든 게이트웨이가 **모델명을 알아서 보정**해 공유 서버의 실제 모델로 연결합니다. 추론 누출 차단(think:false)과 답변 길이 상한(num_predict)도 게이트웨이가 처리하므로, 그 옵션들도 그대로 뒤도 안전합니다.

3. 확인 절차

순서대로 복붙하면 됩니다. (민감값 입력 금지 — 5번 안전수칙 참고)

(1) 게이트웨이가 살아 있나 — `/health`

먼저 공유 게이트웨이가 켜져 있는지 확인합니다. 터미널에 그대로 붙여넣으세요.

```
curl http://121.138.151.6:11500/health
```

- 정상이면 **ok** (또는 짧은 상태 JSON)가 돌아옵니다.
- 응답이 없으면 게이트웨이가 꺼져 있거나 네트워크 문제입니다. **강사에게 알려세요.** (이 단계가 안 되면 챗봇도 당연히 안 됩니다.)

(원하면 모델 목록도 볼 수 있습니다.)

```
curl http://121.138.151.6:11500/api/tags
```

(2) 내 앱 기동

백엔드(내 앱) — `app.py` 가 있는 폴더에서:

```
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

프론트(화면) — 같은 폴더에서 새 터미널을 하나 더 열고:

```
python3 -m http.server 5500
```

그다음 브라우저에서 `http://127.0.0.1:5500/index.html` 을 엽니다. (주의: `index.html` 을 파일로 직접 더블클릭하면 CORS로 막힙니다. 반드시 `:5500` 주소로 여세요.)

(3) 대표 질문 1개로 테스트

화면 입력창에 아래를 넣어 봅니다.

“약제부 어디 있어?” → 기대: 본관 2층, 259제곱미터 + 근거 칩(약제부) “물 끓이는 설비 있는 곳?” → 기대: 키워드가 안 맞아도 의미로 기계실(본관 지하6층) 을 찾아 답 “면적 합이 가장 큰 부서는?” → 기대: 시설팀(234제곱미터) (백엔드 집계값 인용)

(화면 대신 터미널로 바로 실행 보고 싶다면)

```
curl -X POST http://127.0.0.1:8000/api/ask \
-H "Content-Type: application/json" \
-d '{"question":"약제부 어디 있어?"}'
```

(4) think(생각 과정) 누출 없는지 확인 ★

- 답변에 `<think> ... </think>` 같은 사고 과정 텍스트나 “사용자가 묻는 것은...” 같은 혼잣말이 섞여 나오면 안 됩니다. 게이트웨이(:11500)를 쓰면 이 부분이 자동으로 걸러져서 깔끔한 최종 답만 나옵니다.
- 혹시 `<think>` 가 보이거나 답이 비어서 나온다면, 십중팔구 주소가 아직 `:11434` (raw)로 남아 있는 것입니다. 2번 절로 돌아가 `app.py` 의 `11434` 가 한 곳도 안 남았는지 다시 확인하세요.

4. 아직 코드가 없다면 — 3주차 핸드아웃 STEP0으로 출발선 맞추기

이 과제는 코드 제출이 없었습니다. 그래도 4주차 실습은 **공통 STEP 0 기본코드** (`app.py` · `index.html`) 만 떠 있으면 출발할 수 있습니다. 절차는 3주차 개인 핸드아웃과 같습니다.

참고 핸드아웃: [3주차_강의_개인별/3주차_강의_01_공간조회.pdf](#) (STEP 0 캐치업 + 프롬프트 1~7)

출발선 점검 (1분):

1. 공통 STEP 0 기본코드(`app.py` · `index.html`)가 있는 폴더로 들어갑니다.
2. 백엔드를 띄우고 살아 있는지 확인합니다.

```
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

다른 터미널에서

```
curl http://127.0.0.1:8000/health
```

→ `{"ok": true}` 류 응답이 오면 백엔드 정상입니다.

3. 여기서 4주차의 핵심을 적용합니다 — `app.py` 안 모델 주소를 `:11500` 으로 바꿉니다(위 2번 절). 그러면 STEP 0 단계부터 이미 게이트웨이 위에서 도는 상태가 됩니다.
4. 그다음 3주차 핸드아웃의 **프론트 1~4**(임베딩 검색 → 출처 → 바이브↔하네스 토글 → 백엔드 집계)를 순서대로 복붙하면 C 조회 챗봇이 완성됩니다.

 임베딩 코사인 계산에 numpy가 필요합니다. 없으면 `pip install numpy` . 💡 3주차와 달라진 점은 **딱 하나** — 모델 주소가 `:11434` 가 아니라 `:11500` 이라는 것뿐입니다. 나머지(프론트/서버 분리, 임베딩 검색, 출처, 토글)는 그대로입니다.

5. 안전수칙 (이 과제 특화)

- **더미·비식별 데이터만.** 공간 데이터는 반드시 **가짜 8칸**(기계실·약제부 등 더미)으로만 실습합니다. 실제 도면·정확한 면적·총별 배치도 같은 **시설 운영자료를 그대로 넣지 마세요.** 화면에 **Demo Mode (샘플 공간 데이터)** 배지를 띄워 두면 헛갈리지 않습니다.
- **보안시설 상세는 금지.** 잠금번호·출입통제 절차·상세 도면·비상금고 위치 같은 **보안 민감 정보는 데이터에도 넣지 말고,** 물어봐도 “보안시설 상세는 안내하지 않습니다”로 거절하게 합니다.
- **환자정보·대외비 입력 금지.** 입력창에 환자 이름·등록번호·진단명이나 대외비 문서를 절대 넣지 마세요. 질문은 “OO실 어디 있어?”, “면적 합이 큰 부서?” 처럼 **공간 단위 조회**로만.
- **공유 GPU는 하나뿐 — 동시성 주의.** :11500 뒤의 모델 서버는 **여러 명이 함께 쓰는 GPU 1대**입니다. 여러 명이 동시에 긴 질문을 몰아넣으면 다 같이 느려집니다. 한 명씩 순서대로 시연하고, 답이 느려도 잠시 기다리세요(서버를 탭하지 말 것).
- **모델 서버 주소·접속법은 외부 공유 금지.** `121.138.151.6:11500` 같은 내부 주소는 교육 내에서만 쓰고 외부로 공유하지 않습니다.